

ASE 420 Tools Installation - Python

Software Engineering and Python

- SE is about rules and **tools**.
- Software engineers solve problems using **tools**.
- In this course, we discuss the **tools** for implementing software design: Python.

| ASE 420 & Python

- We use Python for explaining software design concepts.
- All the code examples are written in Python.
- The course team project uses Python and the Pygame library.
- We use Python for AI examples.

Why Use Python?

- Simple
 - But not necessarily easy
- Packages (Libraries)
 - Using PIP to make and use packages
 - Using venv to isolate packages
- AI-ready
 - Python is the de facto standard language for AI

Install Python

- [Download Python | Python.org](#)
- If you use Mac, use `brew` to install Python instead of pre-installed Mac python.

Venv (Virtual Environment)

| What Problems Does `venv` Solve?

- **Dependency conflicts:** Two projects may need different versions of the same library.
 - Installing both globally can break one project when the other updates a package.
- Venv solves this "Library Hell" complexity.

What Benefits Does **venv** Give?

- **Reproducibility:** Team members can install only the packages needed by a project, instead of relying on an unknown global setup.
- **Project isolation:** A virtual environment gives each project its own Python interpreter and installed packages.

Use venv when you start a project

- Always use `venv` for your project.
- A virtual environment reuses the installed Python interpreter and stores only the packages required by the project, keeping the global Python environment clean and isolated.

Learn how to use venv.

- Read the manual - [Install packages in a virtual environment using pip and venv - Python Packaging User Guide](#)
- When you are using venv, the command line shows the directory of the venv: for example, (p3), at the prompt.

```
(p3) chos5@NKU023R7042 ase420>
```

Starting Venv

- This is an example to start venv on Mac/Linux.

```
python -m venv ~/venv/p3  
source ~/venv/p3/bin/activate
```

- This is an example to start venv on Windows.

```
python -m venv C:\Users\YourUsername\venv\p3  
C:\Users\YourUsername\venv\p3\Scripts\activate.bat
```

- Modern tools (VS Code, PyCharm, `uv`) commonly default to a project-local `.venv/` directory instead of a centralized path like

`~/venv/p3`. Both work, but `.venv` is the more

Python Packages and PIP

- Under the virtual environment, use `pip` to install packages.
- Install pygame for the course project.
- Install jupyter for interactive Python application development.

```
pip install pygame  
pip install jupyter
```

UV

- A modern, fast Python package and project manager from Astral.
 - It is an external tool that provides a faster, unified interface for the same workflow.
 - You can use UV instead of invoking `venv`, `pip`, and related helper tools separately.

What Problems Does UV Solve?

- **Setup complexity:** A new project often requires several commands and tools before code can run.

```
python -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt
```

- UV provides a consistent command-line interface for this workflow:

```
uv venv  
uv pip install -r requirements.txt
```

What Benefits Does UV Give?

A Single Source of Truth for Dependencies

- **TOML** is a readable configuration-file format and stands for "Tom's Obvious, Minimal Language" that UV uses.
- UV can use lock files to install the same dependency versions for every developer and CI job.

pyproject.toml Example

- Modern Python projects use `pyproject.toml` to describe the project in one standard file:
 - Project name, version, and supported Python version
 - Direct dependencies required by the application
 - Build, testing, formatting, and tool configuration

Example:

```
[project]  
name = "my-project"  
version = "0.1.0"  
requires-python = ">=3.11"  
dependencies = [  
    "pygame>=2.6",  
    "jupyter>=1.1",  
]  
  
[tool.uv]  
dev-dependencies = [  
    "pytest>=8.0",  
]
```


Modern UV Project Workflow

- Beyond the `pip`-compatible commands, UV also has its own project commands that manage `pyproject.toml` and `uv.lock` for you automatically:

```
uv init my-project
uv add pygame
uv run main.py
```

Fast Response

- **Slow feedback:** Recreating environments and installing packages repeatedly can slow development and automated tests.
 - UV uses caching and efficient dependency resolution to make these operations faster.
 - Its Rust implementation also helps deliver fast command execution, shortening the feedback loop for developers and CI.

Focus on the problem, not the tools

- You still need to understand environments and dependencies; UV simply gives you a cleaner interface for managing them.
- UV makes setup faster, but it cannot fix unclear or incompatible dependencies.
- Before choosing a command, ask:
 - Which Python and package versions does this project require?

UV - A Better Tool

- UV can create a compatible virtual environment (venv) and manage the packages (pip) inside it.
- UV can use existing `requirements.txt` files, so `pip`-style workflows can be adopted incrementally.
- It also supports lock files, which record the exact dependency versions for reproducible installs.

Prefer to use `pyproject.toml` over `requirements.txt`

- `requirements.txt` is only an installation list, so it does not describe the whole project or clearly separate intended and resolved versions.
- UV uses:
 - `pyproject.toml` to declare what the project needs
 - `uv.lock` to record the exact versions that work
- Keep `requirements.txt` when compatibility with existing `pip` workflows is needed.

UVX - Familiar Ideas from Node.js

- JavaScript developers commonly use package tools such as `npm`, `npx`, and `pnpm`.
- `npx` made it convenient to run a package command without first installing it globally.
- UV provides a similar experience with `uvx`:

```
uvx ruff check .  
uvx black --check .
```

What problem does UVX solve?

- Developers need command-line tools such as linters and formatters, but global installation causes version conflicts and clutter.
- `uvx` runs the required version of a tool on demand, without installing it globally.
- This makes commands easy to share in project documentation and CI.

Why Use `uvx` or `npx` for Applications?

- `uv` and `npm` manage a project's libraries; `uvx` and `npx` run command-line **applications** such as `ruff`, `black`, or `create-vite`.
- Instead of installing and updating a tool globally, run it when needed:

```
uvx ruff check .  
npx create-vite@latest my-app
```

- They automatically download, cache, and select the requested tool version.

UVX/NPX and LLM

- LLM clients use **MCP (Model Context Protocol)** servers to access tools such as files, databases, and web APIs.
- `uvx` and `npx` let the client launch an MCP server directly from its configuration:

```
uvx mcp-server-fetch  
npx -y @modelcontextprotocol/server-filesystem ./workspace
```

Managing Complexity

- The server is downloaded, cached, and versioned automatically without a global installation.
- This gives every developer the same tool command and reduces setup and version-conflict problems.

When you have any issues

- Remember you are **the problem solver**.
- Develop your **debugging skills** as a professional software engineer.
- **Start early** to identify the issues as early as possible

However, ask for help - Don't waste your time

- You don't have much time to lose, especially for the installation.
- Ask other team members who successfully configured their Python environment.
- Use my office hours to visit and ask.
- However, in most cases, AI will give the answers.

Discuss the issue with LLM

- Copy the error message to LLM.
- Use multiple LLMs when one LLM doesn't give you the correct answers.
- In the AI age, your time is more important than AI time.
- **Learn to collaborate with LLMs** rather than blindly following their output; this is too dangerous.